

Instituto Tecnológico de Buenos Aires



Autómatas, Teoría de Lenguajes y Compiladores - 72.39

Quizzy

Grupo G99

Trabajo práctico especial

Integrantes:

- Othatceguy, Manuel (64671)
- Sala, Bartolomé (64556)
- Tiscornia, Gregorio (64087)
- Zakalik, Tomás (63719)

Fecha de entrega: 28/11/2025

Índice

1. Introducción.....	2
2. Modelo Computacional.....	3
2.1. Dominio.....	3
2.2. Lenguaje.....	3
3. Implementación.....	6
3.1. Frontend.....	6
3.2. Backend.....	6
3.2.1 Analisis semantico.....	6
3.2.2 Generación de código.....	7
3.3. Adicionales.....	7
3.4. Dificultades Encontradas.....	7
4. Futuras Extensiones.....	8
5. Conclusiones.....	9
6. Bibliografía.....	10

1. Introducción

Este informe documenta la etapa final (Stage III) del proyecto Quizzy: un compilador para un DSL orientado a la creación de quizzes interactivos. El objetivo de Stage III es completar el pipeline del compilador (análisis semántico y generación de código) y producir artefactos finales (un ejecutable que transforma un programa Quizzy en HTML+JS listo para abrir en un navegador), además de reportar las decisiones de diseño, cobertura de pruebas y el estado final de la implementación.

2. Modelo Computacional

2.1. Dominio

Quizzy está pensado para modelar cuestionarios interactivos, preguntas con opciones, respuestas correctas, puntajes, temporizadores y lógica de navegación condicional, de forma declarativa. El compilador toma un archivo de texto con un bloque `quiz { ... }` y produce un HTML/JS que renderiza el cuestionario y valida respuestas en el cliente.

2.2. Lenguaje

El lenguaje de entrada está diseñado específicamente para describir quizzes interactivos mediante una sintaxis declarativa y legible para usuarios no expertos en programación. Su estructura se basa en un único bloque principal `quiz { ... }`, dentro del cual se pueden definir los atributos generales del cuestionario, sus preguntas, reglas de puntuación y mecanismos opcionales de navegación condicional.

En lugar de un formato lineal como XML o JSON, el lenguaje adopta una sintaxis cercana a lenguajes de programación tipo C, con llaves `{}`, asignaciones `=`, listas `[]` y conjuntos `{}` para agrupar elementos. Además, presenta una familiaridad clara con Python, especialmente por el uso de listas entre corchetes y conjuntos entre llaves, reforzando una estructura que los usuarios de ese lenguaje encuentran natural e intuitiva. Esto permite una escritura más fluida y jerárquica, alineada con el modelo mental del usuario: un *quiz* compuesto por preguntas, cada una con su información asociada.

A continuación se muestra un ejemplo sintético:

```
quiz {
  title = "Demo"
  shuffle = true
  time = 3 min
  scoring {
    correct = +points
    wrong = -1
    timeout = -points/2
  }
  questions = {
    question {
      id = q1;
      type = multipleChoice;
      text = "Capital de Francia";
      options = { "Paris", "Roma", "Madrid" };
      answer = "Paris";
      points = 5;
    }
    if (score > 0) next = question("bonus")
    question {
      id = bonus
      type = multipleChoice
      text = "¿Qué lenguaje define Quizzy?"
      options = ["Kotlin", "Python", "Quizzy"]
      answer = "Quizzy"
      points = 5
    }
  }
}
```

Figura 1: Ejemplo de un quiz simple en el lenguaje Quizzy, mostrando la definición de preguntas con sus atributos principales.

Los atributos pueden declararse en cualquier orden dentro del cuerpo del quiz o de una pregunta, siempre respetando la presencia de campos obligatorios como **title**, **questions**, **id**, **type** y **text**. El campo **questions**, en particular, admite tanto una lista (**[...]**) como un bloque con llaves (**{ ... }**), otorgando flexibilidad de escritura.

Las expresiones presentes en reglas de puntuación o en condicionales permiten operaciones aritméticas y comparaciones (**+**, **-**, *****, **/**, **==**, **!=**, **>**, **<**, **>=**, **<=**), combinando literales numéricos, cadenas, booleanos y palabras clave especiales como **score**, **points** y **timeLeft**, que son evaluadas dinámicamente durante la ejecución del quiz en el navegador.

Otro aspecto importante del lenguaje es su soporte para elementos multimedia (imágenes, audios y videos), así como para flujos de navegación no lineales mediante construcciones condicionales del tipo:

```
if (expr) next = question("id") [else next = question("id2")]
```

Esto permite que la experiencia de usuario del quiz sea interactiva y adaptativa según su desempeño.

En resumen, el DSL de Quizzy constituye una representación expresiva y extensible del dominio de cuestionarios interactivos, facilitando la edición manual de quizzes y allanando el procesamiento automático mediante las herramientas de compilación desarrolladas.

3. Implementación

3.1. Frontend

Se implementó con Flex y Bison:

- **Scanner** (`FlexPatterns.l`): tokens para keywords (`quiz`, `questions`, `question`, `type`, `options`, `answer`, `points`, `partialCredit`, `caseSensitive`, `media`, `image`, `audio`, `video`, `if`, `else`, `next`, `score`, `timeLeft`) y literales como `T_STRING_LITERAL`, `T_NUMBER`, `T_DURATION`. También soporta identificadores `:ident` y operadores aritméticos/relacionales.
- **Parser** (`BisonGrammar.y`): construye el AST con nodos `Program`, `Quiz`, `QuestionsBlock`, `Question`, `Media`, `Scoring`, `Conditional`, `Expression`, `Value`, etc.
- El AST resultante se guarda en `compilerState->abstractSyntaxTree` desde `Frontend.c`.

Los tests unitarios de Stage II (`src/test/c/*`) comprueban casos de aceptación y rechazo; se mantuvo compatibilidad con los tests provistos.

3.2. Backend

El backend consta de dos fases principales: análisis semántico y generación de código.

3.2.1 Analisis semantico

- Presencia de campos obligatorios: `title`, `questions` y `id`, `type`, `text` en cada `question`.
- Formato de `time` y `T_DURATION` con unidades válidas (`s`, `min`).
- Unicidad de `id` de pregunta (errores reportados en `src/test/c/reject/*` como `07-id-duplicado`).
- `answer` coherente: si `options` existen, `answer` debe pertenecer a `options` o ser del tipo correcto para `trueFalse` / `shortAnswer`.
- `points` no negativos y con formato numérico.
- Validación de condicionales: `next` apunta a un `id` existente (si no, error semántico).
- Validación de `media`: si se referencia `as label`, el label debe ser único y disponible.
- Tipado y evaluación estática de expresiones complejas que referencian `score` o `timeLeft` (se aceptan pero no se evalúan estáticamente).
- Verificación avanzada de `partialCredit` y multi-answer correctness.

3.2.2 Generación de código

- Emite un HTML funcional que contiene estilos CSS y un script `evaluateQuiz()` que recorre `.question-card` y valida respuestas.
- Soporta renderizado y UI para `multipleChoice`, `trueFalse` y `shortAnswer`.
- Validación en cliente: comparación simple por igualdad y **case-insensitive** (por ahora se ignora `points`, `partialCredit` y `caseSensitive` en la comparación).
- Conteo de `score/total` mostrado al final. Actualmente el HTML trata cada pregunta como 1 punto (no usa `points` desde el DSL).
- `partialCredit` y soporte de respuestas múltiples (multi-answer).
- `caseSensitive`: respetar si el autor lo pide.
- `shuffle` de preguntas y/o opciones
- timer global / por pregunta y respuestas ante `timeout`
- scoring expressions: aplicar fórmulas `correct = score + points`, `wrong = ...` durante la ejecución.
- navegación condicional `next` (if/else next) para flujos no lineales.
- render de media (image/audio/video) con `as label` para referencia.

3.3. Adicionales

Se diseñó el AST con nodos que ya contienen la información necesaria para extender el generador: `Scoring`, `Expression`, `Conditional` ya están representadas en memoria y listas para ser usadas por el backend.

Se incorporaron tests de rechazo que cubren validaciones semánticas típicas (por ejemplo, `13-mal-unidad-tiempo`, `16-bool-con-mayus`).

3.4. Dificultades Encontradas

- Manejo de tipos en expresiones (strings vs numbers vs boolean) y coerciones implícitas.
- Decidir una estrategia para la ejecución de `scoring` dinámico (evaluar en el cliente con JS vs evaluar en tiempo de compilación si es una expresión constante). Se optó por evaluarlo en JS.
- Soporte de `media` (paths relativos vs bundling del recurso) y su correcta carga en el HTML final.
- Conflicto entre tokens de puntuación (`<`, `>`) y operadores relacionales en el lexer. Se resolvió creando tokens específicos (`T_LT`, `T_GT`) en lugar de `PunctuationLexemeAction`. Problema heredado del Stage II que se evidenció en el Stage III al intentar probar los HTML generados.
- Nos encontramos con un desafío al implementar las preguntas "Bonus". Originalmente, pensamos en cambiarles el id agregándoles un prefijo como "bonus_"

para diferenciarlas, pero nos pareció que no era la solución correcta. Para solucionarlo, decidimos no tocar el ID de la pregunta para que los saltos sigan funcionando correctamente. En su lugar, logramos que el generador detecte automáticamente si una pregunta es destino de un salto y le "pega" una etiqueta extra en el HTML (`data-bonus="true"`). De esta forma, el código JavaScript lee esa etiqueta para saber que es una pregunta especial.

4. Futuras Extensiones

A continuación se detallan posibles mejoras futuras para extender las capacidades del compilador y del HTML generado:

1. **Evaluación avanzada de respuestas en HTML**
 - Implementar métricas más ricas que consideren tiempo de respuesta, penalizaciones configurables, o múltiples intentos.
 - Añadir retroalimentación personalizada por opción en preguntas de multiple-choice.
 - Posibilidad de agregar *hints* a las preguntas que se habiliten luego de un tiempo predeterminado si no hay respuesta.
2. **Soporte multimedia mejorado**
 - Opciones para mostrar media solo al responder correctamente o al finalizar. Similar a como *Quizziz* muestra memes si el alumno lo habilita.
3. **Reportes de resultados y analíticas del Quiz**
 - Generar automáticamente un archivo JSON o un dashboard con estadísticas que contenga:
 - Preguntas correctas/incorrectas por usuario.
 - Tiempo tardado por pregunta y tiempo total.
 - Dificultad estimada por pregunta (esto se setearía en el bloque `question`)
4. **Banco de preguntas y generación dinámica**
 - Permitir declarar sets de preguntas opcionales y elegir un subconjunto aleatorio por ejecución.
 - Pesos de probabilidad para selección de preguntas.
5. **Feedback**
 - Explicaciones detalladas para múltiples respuestas incorrectas.
6. **Internacionalización completa del HTML generado**
 - Diccionarios de idioma para botones, resultados y mensajes del sistema.
 - Permitir definir el idioma en el DSL (`lang = "es" | "en"`).
7. **Extensión del DSL para nuevos tipos de pregunta**
 - Completar texto (fill-in-the-blanks) con múltiples huecos.
 - Matching (emparejar elementos) similar a un unir con flechas.
 - Preguntas con imágenes clickeables.
8. **Posibilidad de repetir las respuestas erradas**
 - Similar a como hace *Duolingo*, una vez finalizado el quiz, se hace un repaso por las preguntas erradas del usuario para que pueda intentar nuevamente. Esto no alteraría el score final.

5. Conclusiones

Se entregó un compilador funcional que cubre la mayoría de los requisitos del Stage II y ha avanzado fuertemente en Stage III. El artefacto genera HTML+JS ejecutable que permite realizar quizzes interactivos en el navegador. El diseño del AST y la gramática facilitan la extensión para las features faltantes, que son principalmente de generación y ejecución en cliente, por lo que el trabajo restante se concentra en completar `Generator.c` y la evaluación dinámica de expresiones y reglas de scoring.

6. Bibliografía

Golmar, A. (21 de agosto de 2025). *Proyecto Especial*.

<https://docs.google.com/document/d/1XZy63UwUVHyMCR6p051aer-i9SpbLmoOSGC-gh6Cb-I/>